

Rapid Application Development (RAD) Model

A Modern Approach to Iterative, User-Centric Software Delivery



TEAM 3

Presented By



Aman Singh

Team Member



Abhinandan Prasad

Team Member



Dhruv Singh

Team Member



What is RAD?

A software development model that emphasizes quick prototyping and iterative delivery over rigid upfront planning. RAD was introduced to reduce long development cycles and improve flexibility in building software solutions.



Origin: Popularized by James Martin in 1991, formalizing practices already emerging in fast-paced software shops seeking alternatives to the Waterfall model.



Key Objectives

Speed: Accelerate time-to-market through short, iterative cycles

User Involvement: Active collaboration with stakeholders at every stage

Adaptability: Embrace changing requirements without derailing the project

Iterative Delivery: Continuous improvement through repeated feedback loops

RAD MODEL PHASES

Four sequential phases take a project from concept to deployment

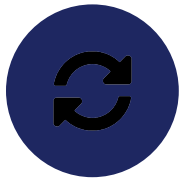


RAD WORKING PROCESS



Planning & Design

Requirements Planning: gather initial requirements and define scope. User Design: develop prototypes with active, ongoing user input.



Prototype & Feedback

Prototype Development: build functional models rapidly. User Feedback: collect input continuously to refine each iteration.



Construction & Delivery

Construction: develop final system components. Testing: verify the system meets requirements. Deployment: deliver the final product to users.

ADVANTAGES OF RAD MODEL



Faster Development

Reduces time-to-market with quick iterative cycles and continuous user feedback.



Reusable Components

Speeds up development; makes it easy to incorporate changes mid-process.



Reduced Development Time

Lower overall effort compared to traditional models; better quality via iterative testing.



High Customer Satisfaction

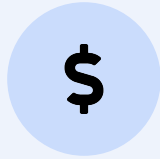
Active user involvement ensures the product meets real needs and expectations.

DISADVANTAGES OF RAD MODEL



Requires Skilled Developers

Teams must be familiar with rapid prototyping; heavy user involvement can be time-consuming to manage.



Higher Initial Cost

Intensive collaboration and specialized tooling raise upfront investment; limited documentation may affect future maintenance.



Not Suited to Large, Complex Projects

Struggles with large-scale efforts that have unclear, unstable, or heavily regulated requirements.

REAL-LIFE APPLICATIONS OF RAD



Banking & E-Commerce

Rapid feature deployment & quick
adaptation to market trends



Healthcare & CRM

Fast integration of workflows &
continuous client customization



Mobile & Booking Apps

Accelerated app releases & adaptability to
dynamic requirements

RAD VS. WATERFALL MODEL COMPARISON

RAD MODEL

Rapid & Adaptive

- Fast, iterative development cycles
- High flexibility — accommodates changes easily
- Continuous & active customer involvement
- Higher initial investment
- Reduced risk through early feedback
- Limited but evolving documentation
- Best for small to medium, dynamic projects

WATERFALL MODEL

Sequential & Structured

- Slow, sequential development phases
- Low flexibility — rigid structure
- Limited involvement, mostly upfront
- Generally lower upfront cost
- Higher risk due to late-stage testing
- Extensive and fixed documentation
- Best for large, well-defined scope projects

WHEN SHOULD YOU USE RAD?



Good Fit When...

- Requirements are known but likely to evolve
- The project is small to mid-sized in scope
- Users/stakeholders can stay actively engaged
- A skilled, modular development team is available
- Time-to-market is a critical business driver



Avoid When...

- The project is very large or mission-critical at scale
- Requirements are heavily regulated or fixed by contract
- Users/stakeholders cannot commit ongoing time
- Strong upfront documentation is a hard requirement
- The team lacks experience with rapid prototyping tools

COFORGE

Thank You!

Questions & Discussion Welcome